

Seminar

Operators - Basic Concepts. Binary Operators.

1. What is operator overloading?

- C++ lets us redefine how **operators** (+, ==, <<, ...) work with **our own classes**.
- The syntax is: <return type> operator<op>(<parameters>).
- Almost every operator can be overloaded. The few that **cannot** are:
:: . .* ?: sizeof
- **Golden rule:** an overloaded operator should do what a reasonable programmer expects: + should add, not subtract.
If the meaning is not obvious, use a named method instead.

2. Two forms.

- **Member function:** the left-hand operand is ***this**.
- **Free (non-member) function:** both operands are explicit parameters.
Declared outside the class (maybe as **friend** so it can access private data).
- When to use which:
 - Operators that **modify** the object (+, -=, *=) → **member**.
 - Symmetric binary operators (+, -, *) → **free function**, so both sides are treated equally.
 - << and >> for streams → **must** be free functions (you don't own **ostream**).
- Example:

— Vector2D.h —

```
struct Vector2D
{
    double x;
    double y;

    // Member form: left operand is *this
    Vector2D operator+(const Vector2D& rhs) const {
        return {x + rhs.x, y + rhs.y};
    }

    // Compound assignment: Always return *this by reference
    Vector2D& operator+=(const Vector2D& rhs) {
        x += rhs.x;
        y += rhs.y;
        return *this;
    }
};

// Free function form: symmetric, preferred for binary +
Vector2D operator+(const Vector2D& lhs, const Vector2D& rhs) {
    return {lhs.x + rhs.x, lhs.y + rhs.y};
}
```

3. Canonical pattern: implement + in terms of +=.

- Write += once, then implement + by copying one operand and applying +=. No duplicated logic.

```
Vector2D operator+(const Vector2D& lhs, const Vector2D& rhs) {  
    Vector2D res = lhs; // copy  
    res += rhs;  
    return res;  
}
```
- The same pattern works for -, *, etc.

4. Comparison operators.

- Always implement != in terms of ==, and >, >=, <= in terms of <. One implementation, no duplication.

```
bool operator==(const Vector2D& lhs, const Vector2D& rhs) {  
    return lhs.x == rhs.x && lhs.y == rhs.y;  
}  
bool operator!=(const Vector2D& lhs, const Vector2D& rhs) {  
    return !(lhs == rhs);  
}
```

5. Unary operators.

- Unary operators take **one operand** - the object itself (*this).
- Unary minus and plus:

```
Vector2D operator-() const { return {-x, -y}; } // negation  
Vector2D operator+() const { return *this; } // identity
```
- Or as a free function:

```
Vector2D operator-(const Vector2D& vec) const {  
    return {-vec.x, -vec.y};  
}
```

6. Why free functions are more symmetric.

- If operator+ is a member, the left operand must be of your class type. A free function allows both sides to be implicitly converted.

```
// If operator+ is a member:  
v + 2.0; // works: v.operator+(2.0)  
2.0 + v; // error: 2.0 has no operator+  
  
// If operator+ is a free function:  
v + 2.0; // works  
2.0 + v; // works
```

7. Operators you should NOT overload.

- && and ||: overloading destroys **short-circuit evaluation**. The built-in versions skip the right operand if the left decides the result. Overloaded versions always evaluate both sides.
- , (comma): very rarely meaningful, almost always confusing.
- Unary & (address-of): overloading it breaks every generic container and algorithm.

8. Type conversion operators.

- Define how your class converts **to another type**.
Syntax: `[explicit] operator <type>() const`.
- **Always use explicit** - without it, the compiler can silently convert your object in unexpected contexts.

```
class Fraction {
    int numerator;
    int denominator;
public:
    explicit operator double() const
        { return (double)num / den; }

    explicit operator bool() const { return num != 0; }
};

Fraction f(1, 3);
double d = (double)f; // explicit cast required
double d = f;         // compile error - prevents accidents
```